Data Management for Big Data

Introduction to databases

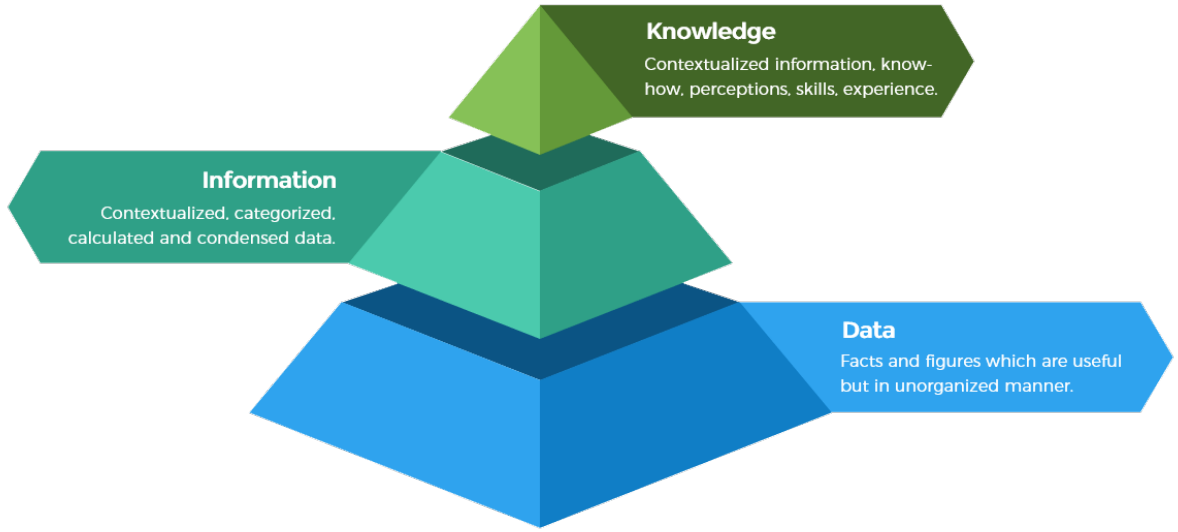
Andrea Brunello
andrea.brunello@uniud.it



Outline

- Different kinds of data
  - *There's a lot of different stuff out there*
- The need for databases
  - *Why can't we simply store files on the disk?*
- Abstraction levels and data models
  - *Things that allow us to define and organize data*
- History of information systems
  - *When did it all start?*

- Regardless of the domain considered, data are the lifeblood of corporate information systems
- **Data** can be defined as a collection of raw, unorganised and unanalysed material relating to a phenomenon (e.g., a sensor connected to an engine outputting decimal numbers)
- Through a processing and **interpretation** phase, the data become meaningful **information** for the recipient (e.g., the sensor is a thermometer, thus, numbers are interpreted as a temperature in degrees °C)
- Combining information, intuition and experience yields **knowledge**, which can be compared with knowledge already acquired and allows us to interpret and guide our actions (e.g., the person schedules a maintenance task on the engine because it is overheating)



Interpretation happens also in Natural Language!

- The word «BAT» alone carries little to no information
- Compare «BAT» paired with the words «CAVERN» and «FLY»; and «BAT» paired with «FIELD», «GAME», and «SWING»
- Note that interpretation requires a human (baseball bat still a baseball bat after humanity disappearance?)



… we interpret also through **context**!

Collecting data in **organized structures**, and **establishing links between them**, it's what makes them informative

Information systems (not to be confused with computer/IT systems) are those systems that support the storage and management of information of an organization for all its purposes

Different kinds of data

- Traditionally, data considered by information systems used to be very simple
- Things started to change in the **2000s**, following the rise of the Internet and its applications, such as social networks
- We can broadly distinguish between the following kinds of data:
  - **Structured** data
  - **Semi-structured** data
  - **Unstructured** data



Structured data can be seen as tabular data, represented by **rows** and **columns**

➔ Each row belonging to a same table has a fixed format, think about an Excel file

➔ For instance, personal data regarding customers

➔ Easy to store and process, but they convey a limited amount of information

	A	B	C	D	E	F
1	Name	Surname	Birth date	Gender	Address	Phone number
2	John	Doe	12/07/74	M	660 North Gonzales Ave. Canyon City, CA 91387	202-555-0123
3	Mary	Jane	15/08/90	F	7772 Shore St. Zion, IL 60099	202-555-0176
4	Darell	Smart	07/03/83	M	2 Pine Ave. Royal Oak, MI 48067	202-555-0197
5	Jessica	Miller	03/11/95	F	7015 Wilson St. South Portland, ME 04106	202-555-0197
6	Belinda	Barton	05/12/67	F	22 Wakehurst Street Jackson, NJ 08527	202-555-0197
7	Ned	Fitzgerald	26/10/77	M	12 Sage Rd. Hope Mills, NC 28348	202-555-0197

Semi-structured data do not have a fixed format; still, there is some kind of structuring within them

- ➔ They make use of **tags** or other markers that allow to organize the content and establish hierarchies within the data
- ➔ For example: XML files, which are a kind of «self-defining» format

```
{
    "EMPLOYEES": {
        "SALES": {
            "648229": {
              "NAME" : "Olivia Johnson"
              "DOB" : "1989-08-08"
            },
            "648666": {
              "NAME" : "Frank Mueller"
              "DOB" : "1985-05-11"
              "MISC" : "On paternal leave from 2019-01-01 until 2020-01-01"
            }
        }
    }
}
```

Unstructured data is **not organized** in any predefined manner

- ➔ Free text
- ➔ Audio and visual materials
- ➔ Difficult to store, index, and analyse



Maura

★★★★☆ **Soddisfatta**

Recensito in Italia il 30 ottobre 2022

Colore: Bianco | Nome stile: Cassettiera per l'armadio | **Acquisto verificato**

Buon designe, facile da montare, o cassetti sono capienti e robusti hanno il fondo rinforzato con un pannello estraibile di cartone rivestito. Il montaggio lo si può seguire dalle istruzioni illustrate nel libretto, il piano d'appoggio robusto, andrebbe migliorata la rifinitura. Leggero da spostare misure dopo il montaggio altezza 72,5 cm Larghezza 46,80 cm profondità 30 cm

Database management system (DBMS)

A *DBMS* is a software system that contains / manages information on a particular domain

- ➔ Collection of **interrelated data** (database)
- ➔ Set of **programs** to manage the data

For example, in the university domain:

- ➔ **Data**: students, professors, courses, …
- ➔ **Programs**: add a new student, modify the salary of a professor, enrol a student to a course, …

Databases can be very large, and they touch many aspects of our life!



In the early days, data-centric applications were built by means of:

- ➔ **Files**, e.g., in “csv” format
- ➔ **Programs**, specifically designed to access such files

First commercial relational DB in the early 80s, after 10 years of research and development

- ➔ So, the need for DBMSs was seen from the beginning of Computer Science

Vehicles.txt

```
Plate,Year,Manufacturer,Model
AX252KF,2015,Ford,Mustang
BK356PQ,2020,Dodge,Charger
FH665HG,2019,Chevrolet,Corvette
CD124LP,2021,Chevrolet,Camaro
```

Belongs_to.txt

```
Plate,Customer_name,Customer_surname
AX252KF,John,Doe
BK356PQ,Mary,Jane
FH665HG,Jessica,Miller
CD124LP,Ned,Fitzgerald
```

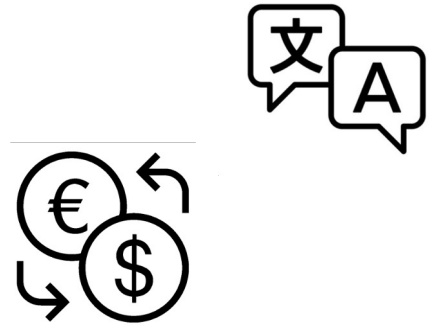
Customers.txt

```
Name,Surname,Birth_date,Gender
John,Doe,1974-07-12,M
Mary,Jane,1990-08-15,F
Darell,Smart,1983-03-07,M
Jessica,Miller,1995-11-03,F
Belinda,Barton,1967-12-05,F
Ned,Fitzgerald,1977-10-26,M
```

Drawbacks of relying on files and programs

Data **redundancy and heterogeneity**

- ➔ Multiple file formats and programming languages
- ➔ Duplication of information in different files
  - ◆ Waste of disk storage
  - ◆ Possible data inconsistency!



Difficulty in **accessing data**

- ➔ Need to write a new program to carry out each new task
- ➔ Low interactivity



Drawbacks of relying on files and programs

Constraints management

- ➔ Domain constraints (e.g., car value >= 0) become «buried» in program code rather than being stated explicitly
- ➔ Hard to add new constraints and manage existing ones



Atomicity of updates

- ➔ Failures may leave the system in an inconsistent state
- ➔ E.g., a transfer of funds from one account to another should be either carried out in its entirety or not happen at all



Drawbacks of relying on files and programs

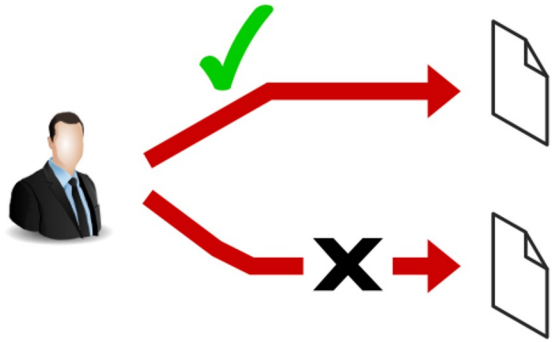
Concurrent access by multiple users

- ➔ What happens if two users try to access and modify the same data at the same time?
- ➔ E.g., two users booking the last airline ticket



Security problems

- ➔ Grant users access to some, but not all, data



(Relational) DBMSs provide solutions to all these problems!

Characteristics justifying the use of a DBMS

- **Large** amounts of data
- **Global** data
  - They are relevant for a vast array of users and applications
  - The database is typically not tied to a particular user or application,
- **Persistent** data
  - The database is there independently from the interacting users or applications
  - Data lives on its own unlike, for instance, program variables

- ➔ To be capable of dealing with all the previous stuff we mentioned, a DBMS must be a very **complex piece of software**
- ➔ To tackle the complexity inherent in their development and usage, DBMSs make use of **abstractions**
- ➔ Abstractions are a fundamental concept in Computer Science
- ➔ **They keep complexity under control** allowing the user to focus, at each time, on what is important, leaving out irrelevant details that would generate confusion



It is like looking at a painting from different distances:

➔ **looking closely** you focus on the brushing technique and how the colors are laid down from a "technical" standpoint

➔ **moving farther away** you appreciate and understand the overall scene, ignoring the brushing details… you see a dog running across a field on a sunny day, you do not care about how its ears are painted



➔ In practice, for what concerns the database realm, abstraction involves **stratifying the DBMS**, to allow for its easier development and management

➔ This means that the DBMS functionalities are arranged into **different hierarchical levels**

- each level encapsulates/hides the details it deals with,
- it offers its functionalities to the higher levels,
- and it can rely on the functionalities made available by the lower levels, without caring about their implementation (**abstraction**)



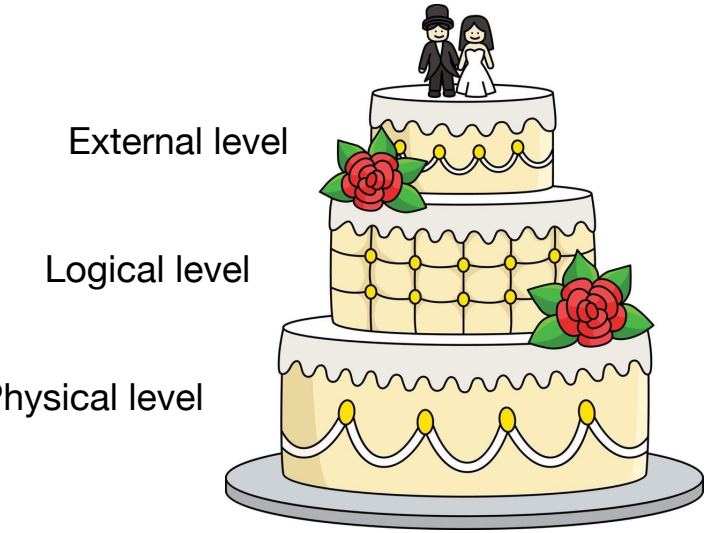
From lowest (bottom) to highest (top):

➔ **Physical level**: *how* data is stored on the disk (data structures, file arrangement, …)

➔ **Logical level**: *what* data is stored/represented in the database and their relationships

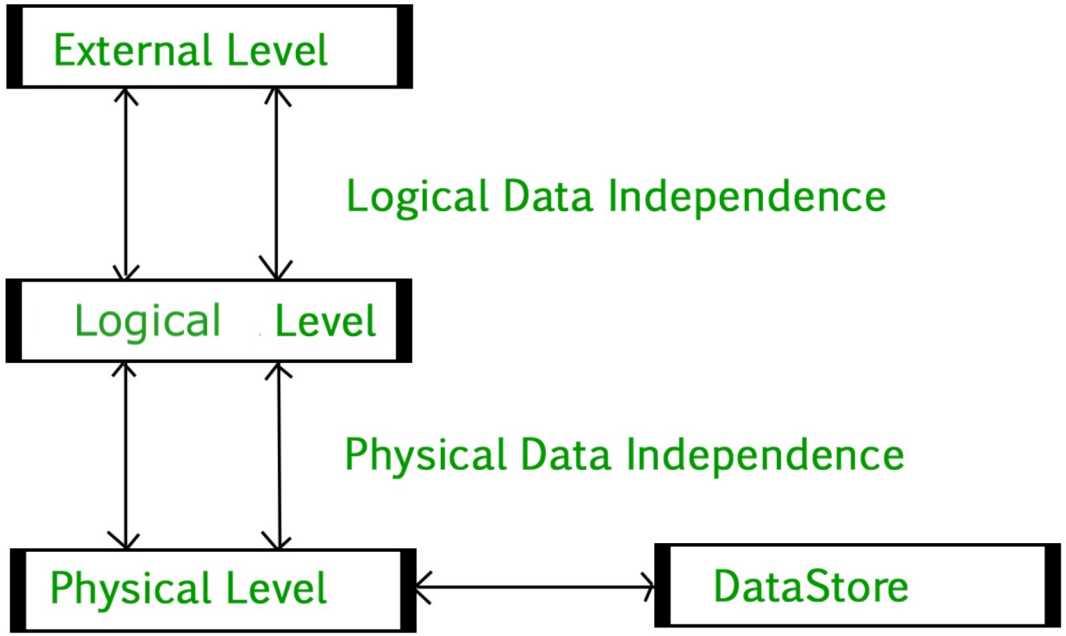
- describes an entire database in terms of a small number of simple structures
- e.g., data is stored within tables, or graphs
- they are *tables/graphs*, i.e., an abstract representation… we do not care about how they are stored on the disk (the physical level deals with it)

➔ **View/External level**: it manages the selective presentation of information to users and programs (optional)



Physical data independence: ability to make changes to the physical level without having to modify the logical one

Logical data independence: by means of *views,* ability to make changes to the logical level without affecting the external one (just the *translation* between the two)



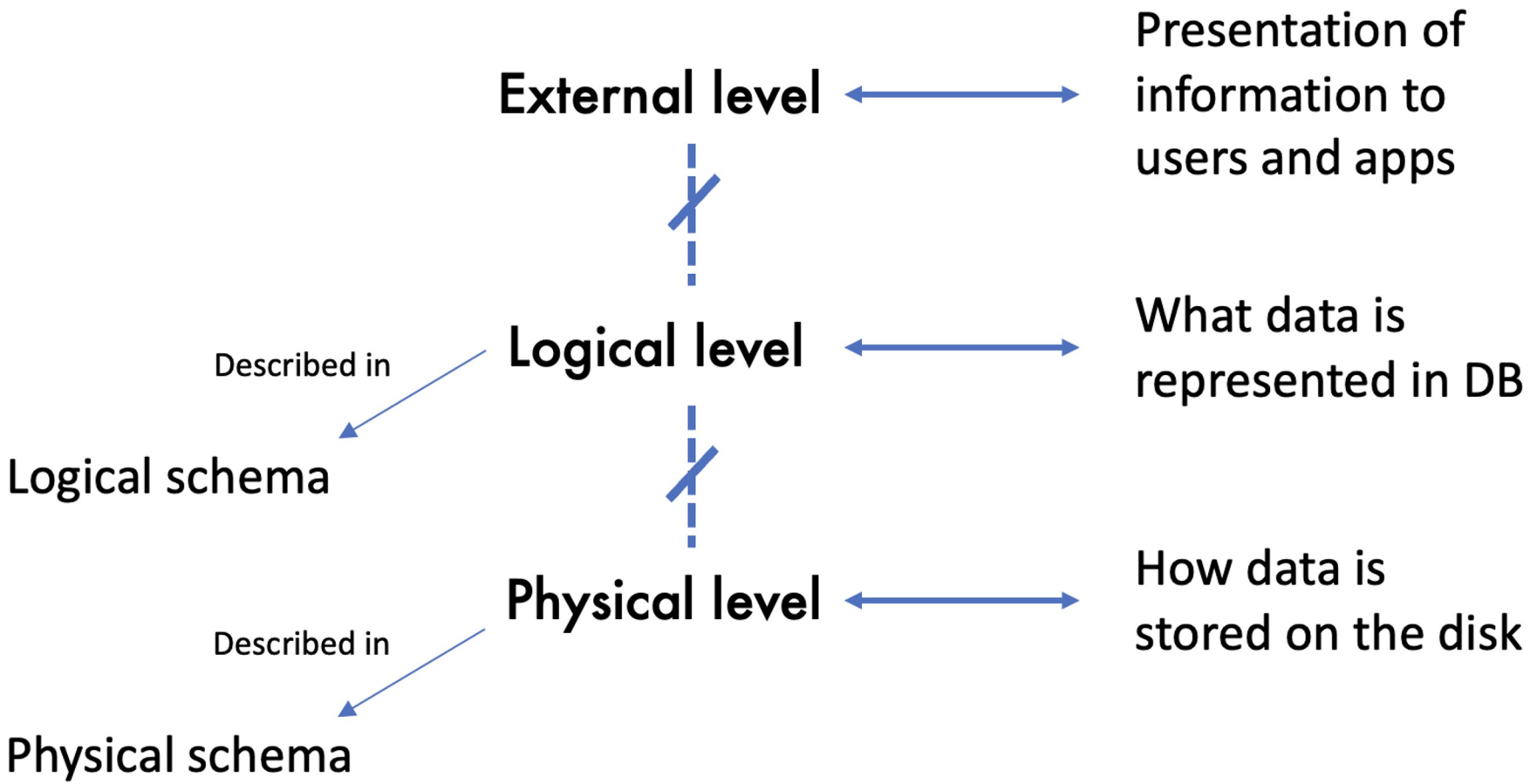
… Just like you shouldn't have to learn a new way to use your car because the mechanic swapped your V6 for a V8 engine



The levels of abstraction of a specific database are described by means of *schemas*:

- ➔ **Physical schema**: it lays out how data are stored physically on a storage system in terms of files and data structures
- ➔ **Logical schema**: it encodes the overall logical structure of the database
  - ◆ E.g., the database consists of a set of tables that are designed to store information about a set of customers (each characterized by name, address, and phone number) and their accounts in a bank;
  - ◆ the relationships among customers and accounts;
  - ◆ and, some constraints over them

Recap: levels and schemas



➔ To describe the logical schema, a small number of simple structures is used (data model)

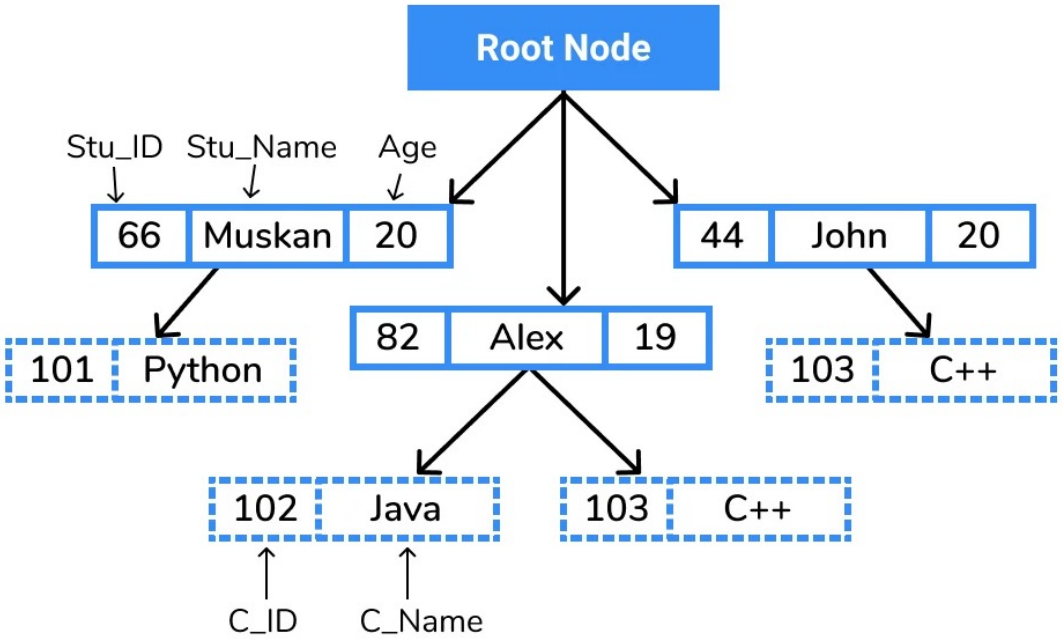
➔ A **data model** is a set of tools (like an alphabet/language) for describing

- Data (and its structure)
- Data relationships
- Data constraints

➔ There exist several logical data models, including

- The **relational data model**, which considers rows and tables (records and relations)
- Graph data model, that deals with nodes and vertices (important for AI)
- Object-based data models
- Hierarchical and network models

Hierchical data model



Records physically point to other records; only «parent-child» relationships are allowed

No clear distinction between logical and physical levels: the physical arrangement influences the logical structure, and vice-versa; also, the user must know how to navigate the structure to reach a piece of data

Evolution: network model structure (*The Programmer as a Navigator*, Bachman, 1973)



Tables to represent data and relationships among the data

Logical and physical levels are clearly separated; instead of encoding traversal logic, users can express desired results, and the DBMS figures out access paths

Students

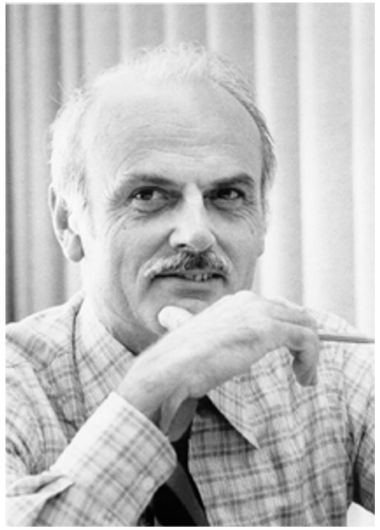
ID#	Name	Phone	DOB
500	Matt	555-4141	06/03/70
501	Jenny	867-5309	3/15/81
502	Sean	876-9123	10/31/82

Takes_Course

ID#	ClassID	Sem
500	1001	Fall02
501	1002	Fall02
501	1002	Spr03
502	1003	S203

Courses

ClassID	Title	ClassNum
1001	Intro to Informatics	I101
1002	Data Mining	I400
1003	Internet and Society	I400



Ted Codd

The graph data model uses nodes and vertices to store the data.

The relational and the graph data models are equivalent pertaining to their data representation capabilities.

Sales

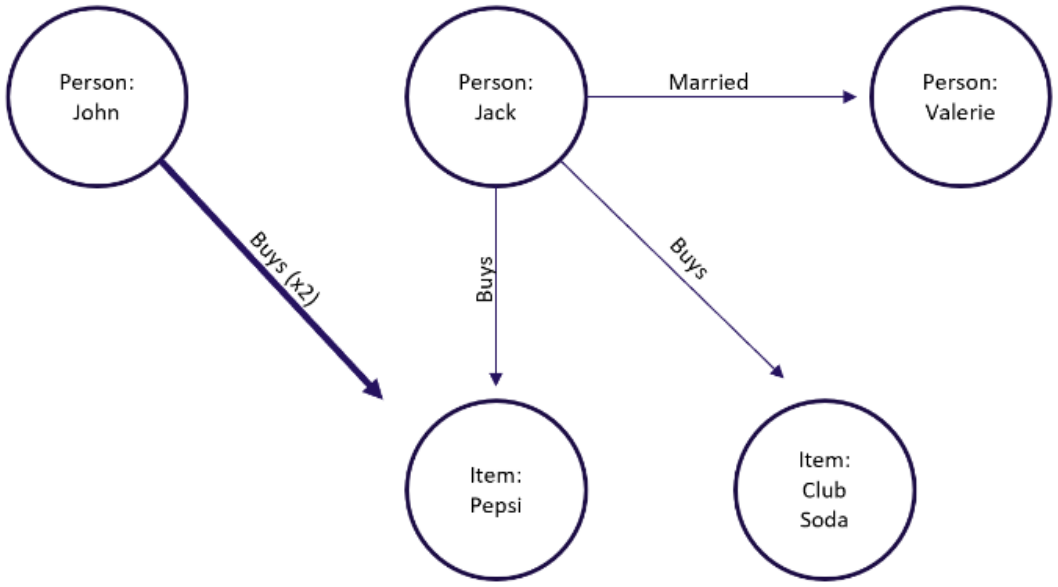
Customer	Item	Time
0001	1A	20:34
0001	1A	21:15
0003	2A	21:16
0002	1A	21:16
0002	5C	

Inventory

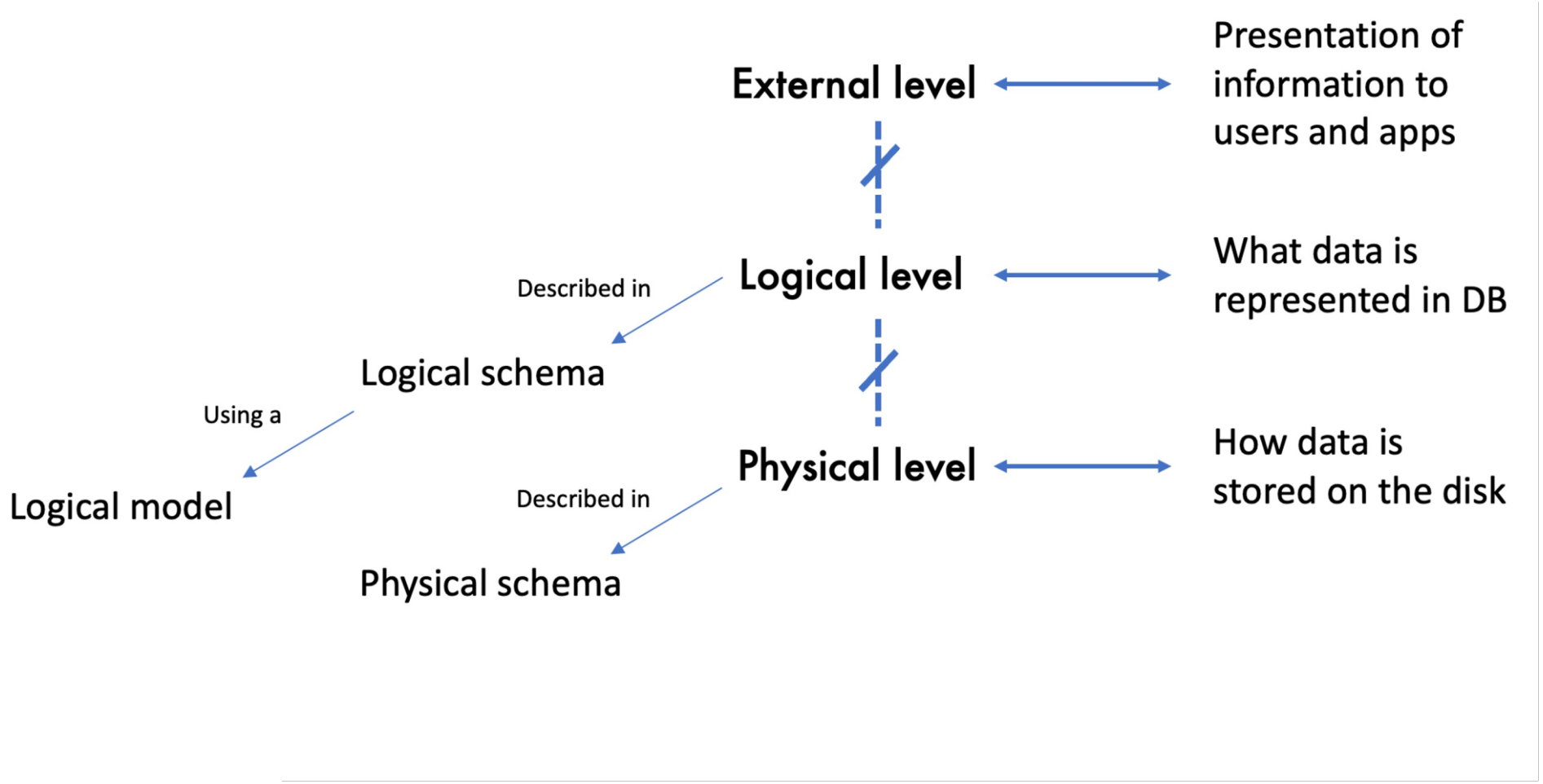
Description	SKU
Pepsi	1A
Club Soda	2A
.	.
.	.
Diet Coke	5C

Customer

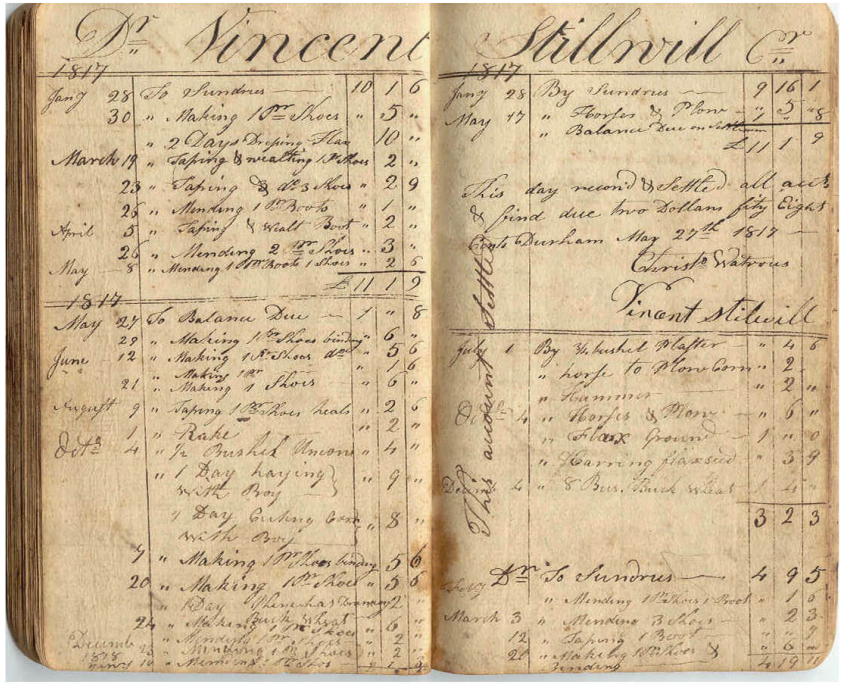
Name	CustID
John	0001
Jack	0002
Ted	0003
Ken	0004
Valerie	0005



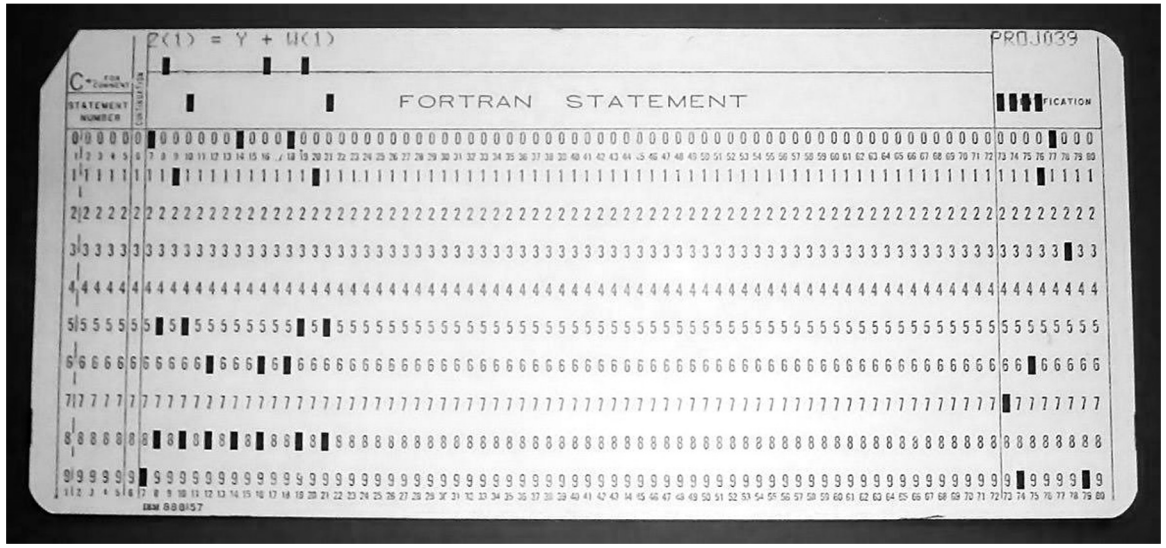
Recap: levels, schemas, and models



- ➔ Every kind of organization has an information system which organizes and manages data to support relevant business goals
- ➔ Information systems **pre-date computers** and even electricity, and are not necessarily automated systems
- ➔ For instance, think about accounting and bookkeeping, or the civil registry



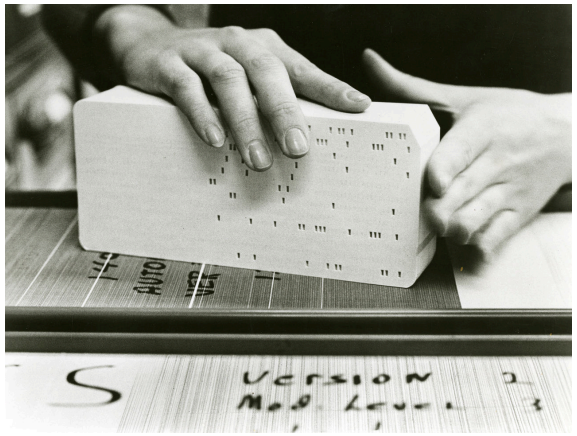
- ➔ 1950s and early 1960s see the first computer information systems
- ➔ Data processing done using magnetic tapes for storage: it provided only **sequential access**
- ➔ Punched cards for input
- ➔ Many dedicated hardware as many formats available
- ➔ Support for low-interaction tasks such as payroll management





These were the golden years of «batch processing»…





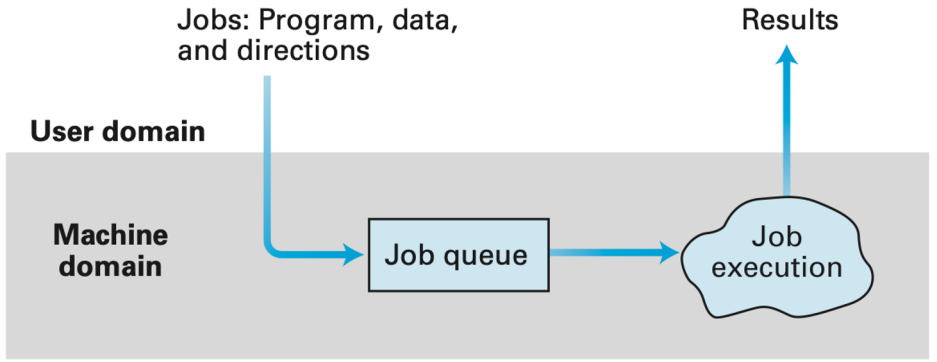


5 MB of data on 62,500 punch cards in 1955.

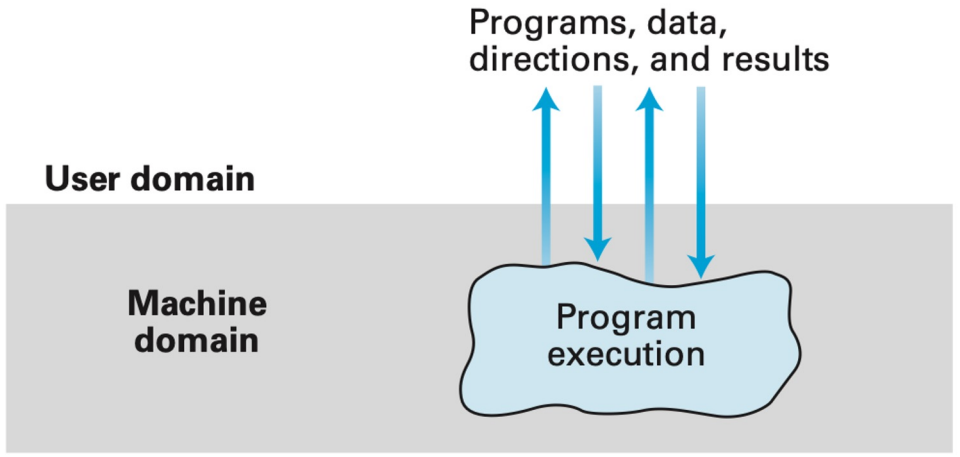


These were the golden years of «batch processing»…

Batch processing

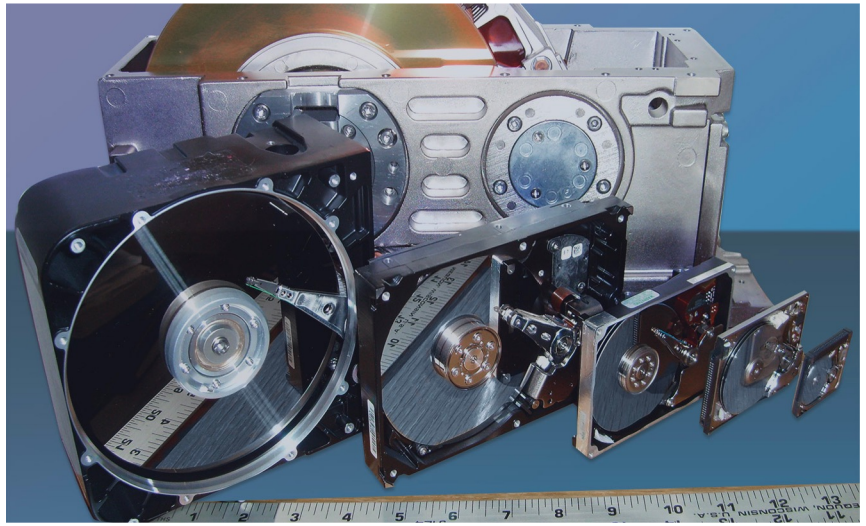


Interactive processing



- ➔ Mid 1960s / early 1970s
- ➔ Hard disks allow for a **direct access** to data
- ➔ The time to locate a record is now measured in milliseconds
- ➔ New, more complex data structures were developed, such as lists and trees to be stored on disk
- ➔ Along with the direct access to data, it came a new type of software known as a **database management system** (DBMS)





- With DBMSs it was easier to access and manage data on a disk; moreover, the DBMS took care of tasks such as storing data, indexing data, governing access rights, and so forth

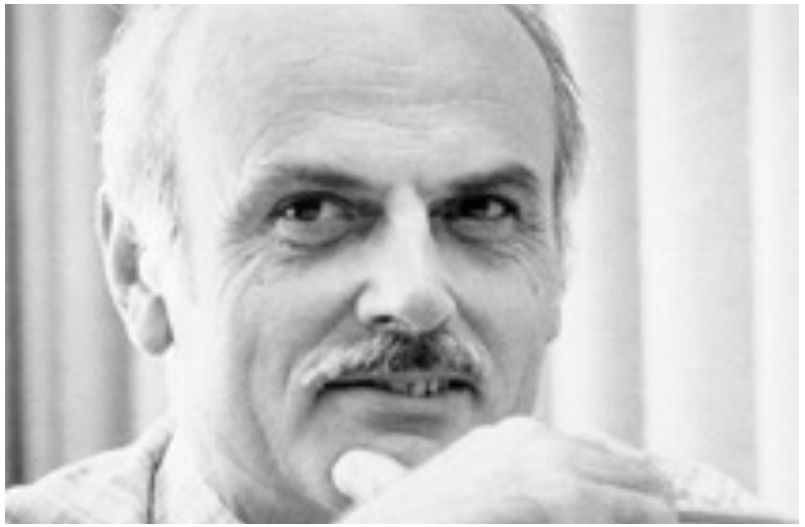
- By the mid-1970s, a new, interactive way to manage the data came to be known as **online transaction processing** (OLTP)

- **Interactive applications** like bank teller systems and manufacturing control systems became increasingly widespread

- At this point, network and hierarchical data models were of widespread use

Meanwhile, Ted Codd defines the **relational data model**:

- ➔ He would later win the ACM Turing Award for his work
- ➔ Points of strength: simplicity and hiding of physical details
- ➔ IBM Research begins work on *System R* prototype
- ➔ UC Berkeley begins work on *Ingres* prototype
- ➔ They both still exhibit computationally worse performance with respect to previous network and hierarchical data models



1980s:

- ➔ Relational database prototypes evolve into commercial systems, overthrowing previous data models
- ➔ **SQL** becomes an industrial standard: programmers can now work at the logical level, without worrying about query optimality and the actual location of data
- ➔ **Parallel and distributed** database systems, on dedicated architectures
- ➔ Object-oriented database systems: they use the same model as object-oriented programming languages



- Around 1990 Wal-Mart retail corporation began to achieve wide acclaim for its mastery of supply chain management
- Behind this success there was a **data warehouse**, and a yet another way of interacting with data, called **online analytical processing** (OLAP)
- Data was collected from all its point-of-sales systems to achieve unprecedented insight into the purchasing habits of its 100 million customers and the logistics guiding its 25,000 suppliers
- Wal-Mart's data warehouse was the first commercial Enterprise Data Warehouse to reach 1 terabyte of data in 1992



- Around 2000s, the types of data stored in database systems evolved rapidly, pushed by an ever increasing usage of the Internet and multimedia (**Big Data**)
- The variety of new data-intensive applications led to **NoSQL** systems, which gave programmers greater flexibility to work with new types of data
- **Distributed** storage and computing frameworks, capable of running on commodity hardware were developed, such as Hadoop
- To allow for the interchange of information between systems, formats such as **XML** and JSON became of widespread usage
- **Data Mining** applications started to emerge
- Most of large/medium sized organizations started to rely on **Decision Support Systems** (DSSs)

- A decision is the selection of a course of action from a set of alternatives
- A **DSS** recommends such an action by offering managers information upon which to build ideas so to come up with the final choice
- A **Decision Management System** (DMS) is an *action-oriented* evolution of a DSS
- It makes one step more and takes decisions without human intervention based on known information and a set of coded business rules or **artificial intelligence models**
- Of course, not all judgments may be automated (strategic vs operational decisions)

➔ From 2010s, more and more companies began to shift from an on-premise management of their DSSs / DMSs to **cloud solutions** (e.g., offered by Microsoft or Amazon)



➔ Another important revolution in the field, starting in 2023, is the advent of **Large Language Models** (LLMs), that promise to make the design and advanced usage of databases accessible even to non-experts

References

- A. Silberschatz, H.F. Korth, S. Sudarshan, *Database system concepts*, 7th Edition, 2020

- W.H. Inmon, *Building the Data Warehouse*, 4th Edition, 2005